

BSIDES KRAKÓW 2026

Runtime Security: From Containers to Virtual Machines with eBPF

Same policy. Same binary. Same node. Different kernel — and your sensor doesn't know it.

A little about me

Renato Rodrigues — Brazilian, living in Poland for 7+ years

Senior Platform Engineer at Surfshark — Platform Engineering Team

10+ years in Open Source and Cloud Native

Founder, LyraKube Labs and Bifrost Systems

Everything in this talk runs on my private DC — nothing is staged

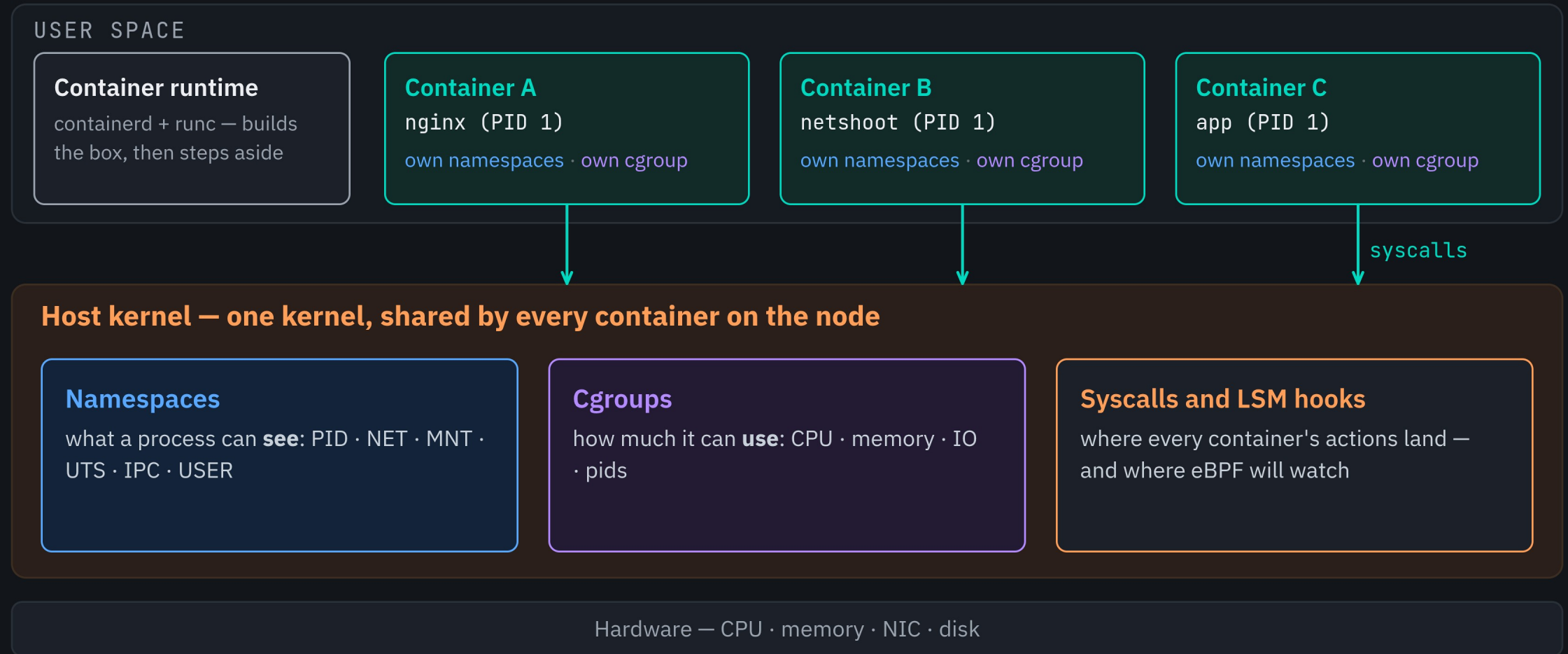


THE PROBLEM

Most real-world attacks happen after deployment — when the workload is already running

- CI/CD scanning cannot predict runtime behavior
- Applications behave differently in production
- Attackers exploit runtime gaps: network, exec, misconfiguration

A container, in one slide — where each piece lives



One kernel, many containers

Container A

a process

Container B

a process

Container C

a process

isolated by namespaces and cgroups

Namespaces

PID · NET · MNT · UTS — logical isolation

Cgroups

CPU · memory · IO limits

all sharing

Host Kernel

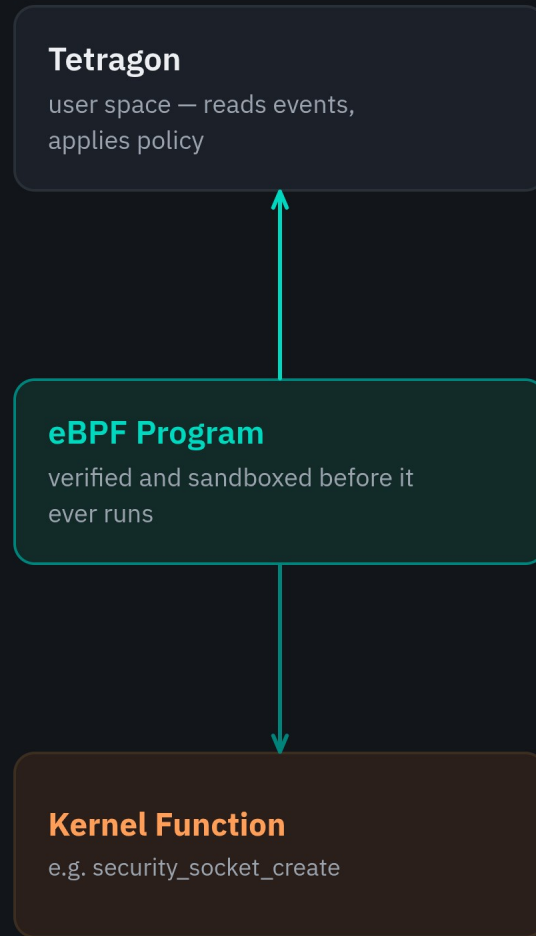
one kernel, shared by every container on this node — a weaker boundary than a VM, by design



The kernel becomes a programmable observability layer

- Runs small, verified programs directly inside the Linux kernel
- Hooks syscalls, observes network traffic, tracks process behavior
- No application changes, very low overhead, deep visibility

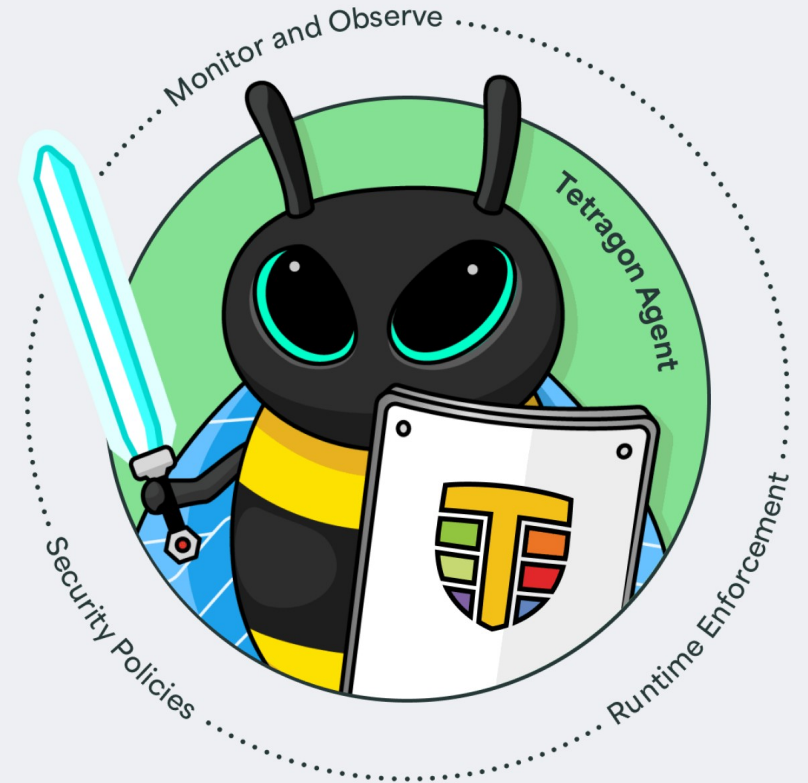
A program running inside the kernel



TETRAGON

eBPF-based runtime observability and enforcement

- Monitors process execution, syscalls, and network connections
- Not just detection — enforces policy in real time, as events happen
- Runs as its own agent (tetragon-agent), separate from Hubble



One agent, several hooks



Tetragon ≠ Cilium

Cilium	Tetragon
Network policy	Runtime and process policy
A CNI	Runs standalone — no CNI, no Kubernetes required
Works alongside any CNI	Treats Cilium as a first-class citizen, but does not depend on it

Because Tetragon needs neither Kubernetes nor a CNI, the identical sensor and policy can run inside a bare VM.

Two planes, one kernel

	Cilium cilium-agent · the cluster's CNI	Tetragon tetragon · standalone agent
Asks	“Is this connection allowed?”	“Should this process be doing this?”
Enforces	Network policy — who may talk to whom, L3 to L7	Runtime policy — what a process may execute, open or call
Attaches via	tc · XDP · socket hooks — packets and connections	kprobes · tracepoints · LSM — exec, syscalls, socket_create
Needs	A Kubernetes cluster — it is the network	Nothing else — any CNI, or no Kubernetes at all
↓ ↓ Linux kernel — both compile to eBPF and run here, side by side independent programs on independent hooks: Tetragon works with Cilium, but never requires it		

LIVE DEMO – PART 1

The container baseline

Where Tetragon is watching from

Attacker Pod

netshoot — nmap, curl, tcpdump

Victim Pod

nginx — allowed traffic still works

Tetragon Agent

DaemonSet, one pod per node

all observed at

Host Kernel — k8s-worker-0

Every syscall from the Attacker Pod and the Victim Pod passes through here — where Tetragon's eBPF hooks are watching. Attacker and victim, and the sensor itself, all share this one kernel.

Same attack, before and after

No policy loaded

`nmap · curl · tcpdump`

All succeed. Connect events stream by, unblocked.

Policy loaded

`nmap · curl · tcpdump`

Every one dies at SIGKILL — same binary, same node

THE SHARPEST DETAIL

The earliest hook wins

tcpdump dies at `security_socket_create` — before a single packet exists, before any network activity at all.

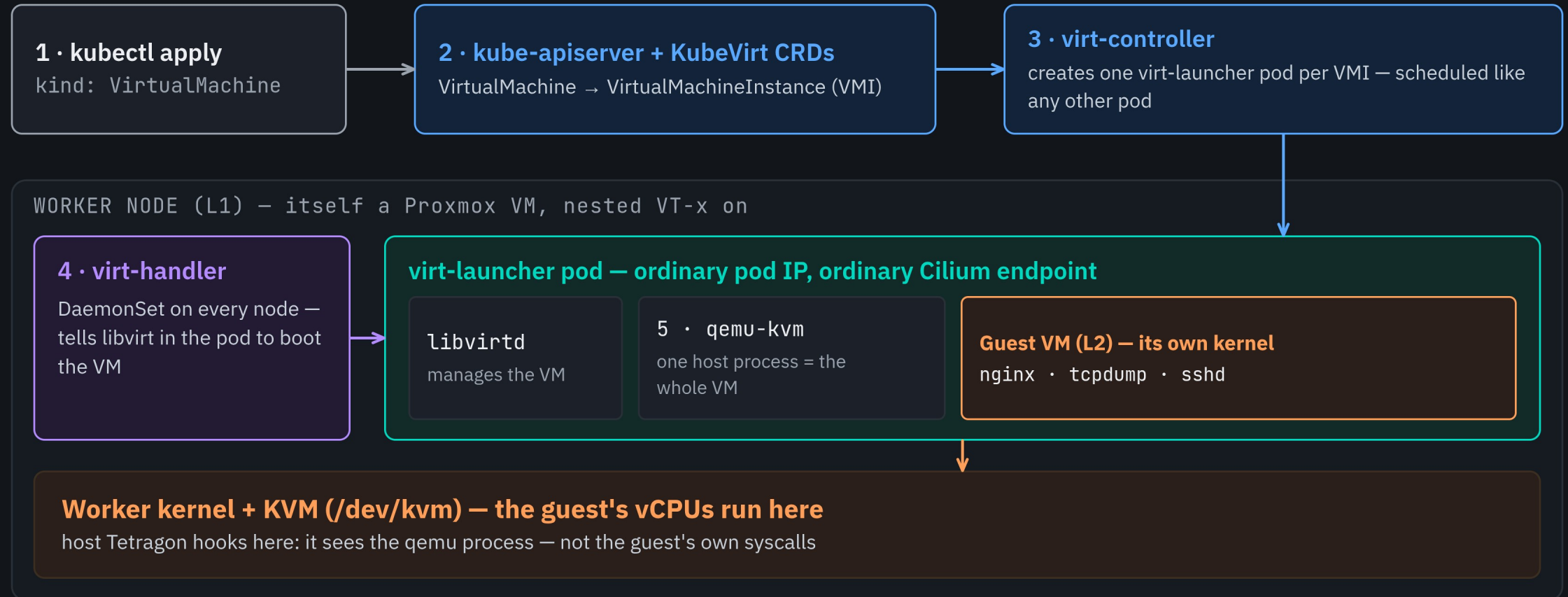
The process never even got the chance to try.

LIVE DEMO – PART 2

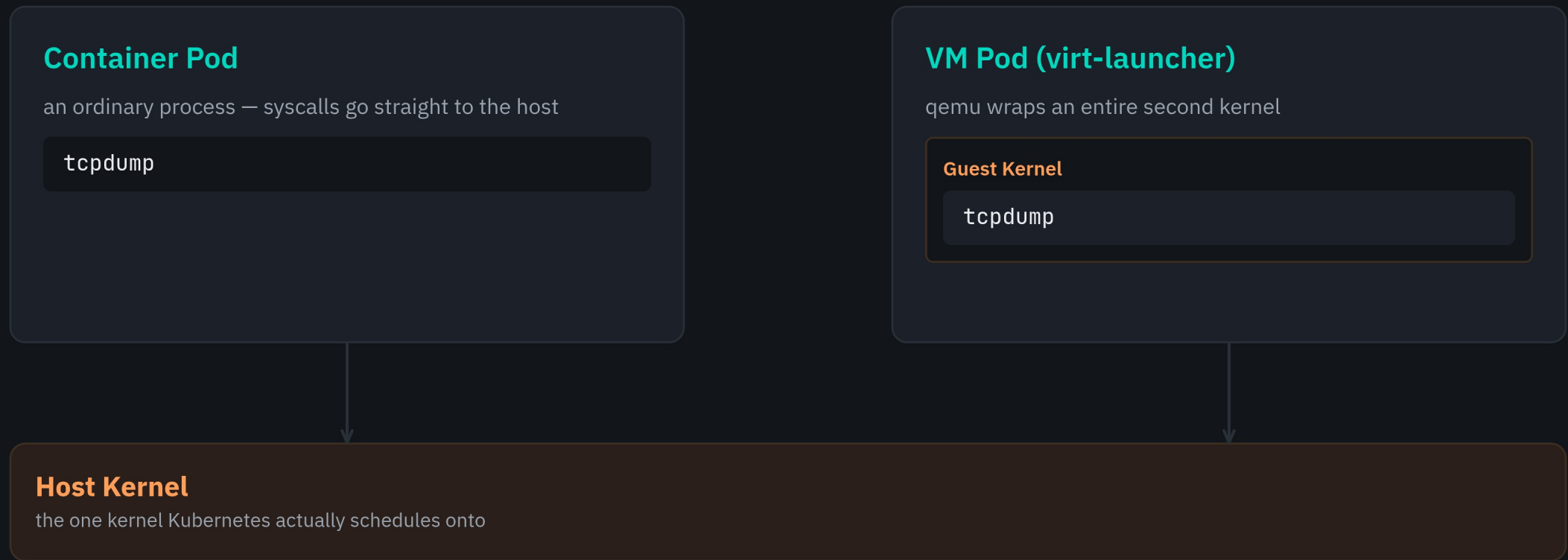
But what about virtual machines?

BEAT 1

A VM is just a workload — how KubeVirt runs one



Same node, two shapes



BEAT 2

Same attack. Same policy. Same node.

Pod

137

killed

VM

124

survives

Only the kernel differs.

PHASE 3

Same policy, different result

Container Pod

same policy, same kernel — killed instantly

```
tcpdump - SIGKILL
```

```
exit 137
```

VM Pod (virt-launcher)

same policy — but a different kernel entirely

Guest Kernel

```
tcpdump - running
```

```
exit 124
```

Host Kernel

the identical kill policy is loaded on this same node for both

BEAT 3

The host sensor is structurally blind

Scoped to the exact pod running the VM, the host's event stream shows nothing during the guest attack — not quiet. Absent.

There's no shared kernel here. Unlike a container namespace, nothing crosses the boundary at all.

The boundary the host can't see past

Container Pod

host Tetragon sees this process directly

```
tcpdump - visible
```

VM Pod (virt-launcher)

host Tetragon sees qemu — nothing past it

Guest Kernel — no shared kernel, nothing crosses

```
tcpdump - invisible to the host
```

Host Kernel — where the host's Tetragon actually hooks

it stops exactly here — there is no syscall crossing into the guest to hook

BEAT 4

Move the sensor into the guest

The same policy file, loaded inside the VM. Now it dies too — and the guest's own Tetragon prints the identical event style.

```
process nginx-vm /usr/bin/tcpdump -i any -c 3  
exit nginx-vm /usr/bin/tcpdump -i any -c 3 SIGKILL
```

A second sensor, one layer up



BEAT 5 – THE CATCH

There's no free lunch

- A host sensor is outside the container's reach. An in-guest sensor is inside the attacker's blast radius
- Root in the guest can stop its own sensor — so the sensor itself must be monitored
- The host still owns network identity, policy, and the hypervisor boundary

Two different trust boundaries

Container Pod

no in-container sensor to tamper with — the host was always outside its reach

VM Pod (virt-launcher)

Guest Kernel

Tetragon Agent

root in the guest can kill this sensor too — it must be monitored itself

Host Kernel — still authoritative here

network identity, network policy, and the hypervisor boundary itself never move into the guest

Process-level enforcement follows the workload only as far as the kernel boundary

VMs break that assumption silently — unless you deliberately move the sensor.

If you run mixed VM and container clusters: check whether your runtime tooling actually reaches into the guest. Don't assume it does just because Kubernetes scheduled it.

Thank you!